

Automated PQC Compliance and Auditing in CI/CD Pipelines

Day 30 | Category: PQC Adoption & Industry Status

Date: 2026-09-23 |

DAILY MONITORING

Automated PQC Compliance and Auditing in CI/CD Pipelines

As the transition to Post-Quantum Cryptography (PQC) accelerates, ensuring that software development practices adhere to new cryptographic standards is paramount. Integrating PQC compliance and auditing directly into Continuous Integration/Continuous Deployment (CI/CD) pipelines provides a scalable and automated solution to manage this transition, enforce security policies, and prevent the deployment of vulnerable code.

The Role of Automation in PQC Transition

Manually auditing every piece of code for PQC compliance is impractical in modern development environments. Automation within the CI/CD pipeline is the key to enforcing standards at scale. This involves several layers of checks:

- **Static Analysis (SAST):** Tools can be configured to scan code for hardcoded or outdated cryptographic algorithms (e.g., RSA, ECC) and flag them for replacement with approved PQC standards like ML-KEM and ML-DSA.
- **Dependency Scanning:** Automated scanners can check third-party libraries and dependencies to ensure they are using PQC-compliant versions. This is critical for managing supply chain risk.
- **Dynamic Analysis (DAST):** During runtime tests, automated tools can inspect network traffic and application behavior to verify that all cryptographic operations (e.g., TLS handshakes, digital signatures) are using the correct PQC algorithms.
- **Policy as Code (PaC):** Using tools like Open Policy Agent (OPA), security teams can define and enforce cryptographic policies as code. The CI/CD pipeline can then query these policies to approve or reject a build automatically.

Real-World Scenario: Blocking a Non-Compliant Build

Imagine a developer accidentally uses a library that still relies on an RSA-based signature for a critical function. The CI/CD pipeline would trigger the following automated workflow:

1. The Static Analysis tool detects the use of a non-compliant cryptographic primitive.
2. The pipeline queries the organization's Crypto-Policy (defined as code), which forbids RSA signatures.
3. The build is automatically failed, preventing the vulnerable code from ever reaching a staging or production environment.
4. A notification is sent to the developer with a clear explanation of the violation and a recommendation to use ML-DSA.

PQC Compliance Auditing Toolchain Comparison

Integrating PQC checks requires a combination of existing and new tools. Here's a look at how a modern CI/CD toolchain can be adapted for PQC compliance:

CI/CD Stage	Tool / Method	PQC Auditing Function	Key Benefit
Commit / Pre-Build	Pre-commit Hooks	Scan for deprecated crypto keywords (e.g., "RSA_sign") before code is committed.	Immediate feedback to the developer.
Build / Test	SAST (e.g., Semgrep, Snyk Code)	Use custom rulesets to identify non-PQC algorithm usage in source code.	Catches vulnerabilities early in the pipeline.
Build / Test	Dependency Scanner (e.g., OWASP Dependency-Check)	Identify dependencies with known vulnerabilities or those not updated to PQC-compliant versions.	Secures the software supply chain.
Deploy / Runtime	DAST / Network Scanners	Intercept and analyze TLS/SSH handshakes to confirm PQC ciphers are in use.	Verifies real-world implementation.

Daily Quiz

1. What is the primary benefit of integrating PQC compliance checks into a CI/CD pipeline?
2. Explain how "Policy as Code" can be used to enforce cryptographic standards.
3. In which stage of the CI/CD pipeline would you use a SAST tool to detect the use of a non-compliant algorithm like RSA?

